

Flight Price Prediction using Machine Learning

A Comparative Study of Five Regression Models on Indian Airfare Data

Name: Asmi Pandey

Roll No: 32

Reg No: 12418814

Section: K3P24MT

Course Code: INT234

Course Title: Predictive Analytics

Teacher Name: Dr. Tanima Thakur

Academic Task Number: 3

Academic Task Type: ASSIGNMENT 3

CERTIFICATE

This is to certify that the project report entitled **"Flight Price Prediction using Machine Learning"** submitted by **Asmi Pandey** (Reg. No. 12418814, Roll No. 32, Section K3P24MT) as Academic Task 3 (Assignment 3) of the course **Predictive Analytics** (Course Code INT234) at Lovely Professional University, Phagwara, Punjab, is a bonafide record of the work carried out by the student under my supervision and guidance during the academic session 2026–27.

To the best of my knowledge, the matter embodied in this report has not been submitted to any other university or institution for the award of any degree or diploma.

_____ _____

Dr. Tanima Thakur
Course Instructor, INT234

Head of School
School of Computer Science & Engg.

Date: _____

Place: _____

DECLARATION

I, **Asmi Pandey**, student of Lovely Professional University, Phagwara, Punjab, hereby declare that the project report entitled **"Flight Price Prediction using Machine Learning"** is an original work carried out by me as Academic Task 3 (Assignment 3) of the course Predictive Analytics (INT234), under the guidance of **Dr. Tanima Thakur**.

I further declare that the dataset used in this project is publicly available and has been duly acknowledged, and that all other sources of information have been properly cited in the references. This work has not been submitted elsewhere for the award of any degree, diploma or certificate.

Date: _____ _____

Place: _____ **Asmi Pandey**

Reg. No.: 12418814

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my course instructor for Predictive Analytics (INT234), **Dr. Tanima Thakur**, for the valuable guidance, constant encouragement and constructive suggestions provided throughout the course of this project. Their insights into predictive modelling and data analysis helped me shape this work in the right direction.

I am thankful to the School of Computer Science and Engineering and all the faculty members for providing the necessary facilities and a supportive academic environment at Lovely Professional University, Phagwara, Punjab.

I would also like to thank the creators of the *Flight Price Prediction* dataset on Kaggle for making the data publicly available, and the open-source communities behind Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn and XGBoost, whose tools made this analysis possible.

Finally, I am grateful to my family and friends for their continuous support and motivation during the completion of this project.

Asmi Pandey

Reg. No.: 12418814

ABSTRACT

Airline ticket prices in India change frequently and depend on many factors such as the airline, route, class of travel, number of stops, time of departure and, most importantly, how many days before the journey the ticket is booked. For a traveller, this makes it difficult to know whether a quoted fare is reasonable or whether waiting will make it cheaper or costlier. This project applies predictive analytics and supervised machine learning to estimate the price of a domestic flight ticket from its characteristics.

The study uses the publicly available **Flight Price Prediction** dataset from Kaggle, which contains **3,00,153 flight records** scraped from the EaseMyTrip booking website for travel between six major Indian metro cities — Delhi, Mumbai, Bangalore, Kolkata, Hyderabad and Chennai — across six airlines. The data was cleaned by removing redundant columns, 2,213 duplicate rows and 2,226 duration outliers detected using the Inter-Quartile Range (IQR) method, leaving 2,95,714 records for modelling. A detailed Exploratory Data Analysis (EDA) was performed to study the effect of class, airline, booking window, stops, departure and arrival time, route and duration on ticket price.

Categorical variables were converted into numerical form using binary, ordinal and one-hot encoding, resulting in 29 input features. Five regression models were trained on 80% of the data and evaluated on the remaining 20%: **Linear Regression, Ridge Regression, Decision Tree Regressor, Random Forest Regressor and XGBoost Regressor**. Model performance was measured using the Coefficient of Determination (R^2), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE), and validated using 5-fold cross validation.

XGBoost achieved the best performance with an R^2 score of 0.9874 (98.74%), an RMSE of ₹2,557 and an MAE of ₹1,342, followed closely by Random Forest (R^2 = 0.9864) and Decision Tree (R^2 = 0.9811). The linear models reached an R^2 of 0.9096, showing that the relationship between the features and the fare is strongly non-linear. The analysis also found that business class fares are about eight times higher than economy fares and that tickets booked within the last two weeks before departure are significantly more expensive. Finally, the trained XGBoost model was exported and deployed as an interactive web page that predicts fares in real time.

Keywords: Predictive Analytics, Flight Price Prediction, Regression, XGBoost, Random Forest, Exploratory Data Analysis, Machine Learning

TABLE OF CONTENTS

Title **Page No.**

Certificate ii
Declaration iii
Acknowledgement iv
Abstract v
List of Figures viii
List of Tables viii
List of Abbreviations ix

Chapter 1: Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Organisation of the Report 2

Chapter 2: Literature Review 3
2.1 Early Work on Airfare Prediction 3
2.2 Machine Learning Approaches 3
2.3 Gradient Boosting Methods 3
2.4 Research Gap and Contribution 4

Chapter 3: Dataset Description 5
3.1 Source of the Data 5
3.2 Attribute Information 5
3.3 Descriptive Statistics 5
3.4 Composition of the Data 6

Chapter 4: Methodology 7
4.1 Project Workflow 7
4.2 Tools and Technologies 8
4.3 Evaluation Metrics 8
4.4 Cross Validation 9

Chapter 5: Data Cleaning and Pre-processing 10
5.1 Removing Unnecessary Columns 10
5.2 Handling Missing Values 10
5.3 Removing Duplicate Records 10
5.4 Checking Categorical Values 10
5.5 Outlier Detection and Removal 10
5.6 Summary of Data Cleaning 11

Chapter 6: Exploratory Data Analysis 12
6.1 Distribution of Ticket Price 12
6.2 Economy vs Business Class 12
6.3 Airline-wise Analysis 13
6.4 Effect of Days Left before Departure 14
6.5 Effect of Number of Stops 15
6.6 Departure and Arrival Time 15
6.7 Route-wise Analysis 16
6.8 Duration vs Price 16
6.9 Key Insights from EDA 17

Chapter 7: Feature Engineering 18

7.1 Binary Encoding of Class....18
7.2 Ordinal Encoding of Stops....18
7.3 One-Hot Encoding of Nominal Attributes....18
7.4 Correlation with Price....19
7.5 Train-Test Split and Feature Scaling....20
Chapter 8: Model Building....21
8.1 Linear Regression....21
8.2 Ridge Regression....21
8.3 Decision Tree Regressor....21
8.4 Random Forest Regressor....22
8.5 XGBoost Regressor....22
8.6 Hyperparameters Used....23
Chapter 9: Results and Discussion....24
9.1 Model Comparison Table....24
9.2 Discussion of Results....24
9.3 Cross Validation Results....25
9.4 Actual vs Predicted Prices....26
9.5 Residual Analysis....27
9.6 Feature Importance....27
9.7 Sample Predictions....28
Chapter 10: Web-based Demonstration....29
10.1 Model Export....29
10.2 Features of the Web Page....30
10.3 Deployment....30
10.4 Example Use Case....30
Chapter 11: Conclusion and Future Scope....32
11.1 Conclusion....32
11.2 Limitations....32
11.3 Future Scope....33
References....34
Appendix A: Source Code....35

LIST OF FIGURES

Figure 4.1 Workflow of the project....7
Figure 4.2 Train-test split and 5-fold cross validation scheme....9
Figure 5.1 Box plots of price, duration and days left....11
Figure 6.1 Distribution of flight ticket price....12
Figure 6.2 Share of tickets and price by class....12
Figure 6.3 Number of flights and price by airline and class....13
Figure 6.4 Economy ticket price vs days left before departure....14
Figure 6.5 Average price by number of stops....15
Figure 6.6 Economy price by departure and arrival time....15
Figure 6.7 Average economy price by source and destination city....16
Figure 6.8 Duration vs price (random sample of 10,000 flights)....16
Figure 7.1 Example of one-hot encoding for the airline attribute....18
Figure 7.2 Correlation of each encoded feature with price....19
Figure 8.1 Structure of a regression decision tree (illustrative)....22
Figure 8.2 Working of the Random Forest (bagging) regressor....22
Figure 8.3 Sequential boosting in XGBoost — each tree corrects the previous errors....23
Figure 9.1 R^2 score (accuracy) and RMSE of the five models....24
Figure 9.2 Actual vs predicted prices for all five models....26
Figure 9.3 Actual vs predicted prices — XGBoost....26
Figure 9.4 Distribution of residuals — XGBoost....27
Figure 9.5 Top 15 important features — XGBoost....27
Figure 10.1 Architecture of the web-based demonstration....29
Figure 10.2 FareCast India web page — desktop view....30
Figure 10.3 (a) Mobile view of the web page, (b) prediction for a business class flight from Bangalore to Kolkata....31

LIST OF TABLES

Table 2.1 Summary of related work....4
Table 3.1 Description of dataset attributes....5
Table 3.2 Descriptive statistics of numerical attributes....6
Table 3.3 Record count by category....6
Table 4.1 Tools and libraries used....8
Table 5.1 Summary of data cleaning steps....11
Table 6.1 Average economy fare by airline....13
Table 6.2 Average economy fare by booking window....14
Table 7.1 Encoding applied to each attribute....19
Table 8.1 Hyperparameters of the five models....23
Table 9.1 Performance comparison of the five regression models....24
Table 9.2 5-fold cross validation R^2 scores (50,000-record sample)....25
Table 9.3 Sample predictions of the XGBoost model on test data....28

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CSV	Comma Separated Values
CV	Cross Validation
DT	Decision Tree
EDA	Exploratory Data Analysis
IATA	International Air Transport Association
IQR	Inter-Quartile Range
JSON	JavaScript Object Notation
LR	Linear Regression
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Squared Error
OLS	Ordinary Least Squares
R^2	Coefficient of Determination
RF	Random Forest
RMSE	Root Mean Squared Error
XGBoost	Extreme Gradient Boosting

CHAPTER 1
INTRODUCTION

1.1 Background

The Indian civil aviation sector is one of the fastest growing aviation markets in the world. With the rise of low-cost carriers, online travel agencies and a growing middle class, air travel has become a common mode of transport for business and personal trips. Unlike train or bus fares, which are mostly fixed, airline fares are decided using **dynamic pricing** (also called revenue management). Airlines continuously change prices based on demand, remaining seats, competition, season, time to departure and many other factors.

Because of dynamic pricing, two passengers on the same flight may pay very different amounts depending on when and how they booked. For customers this creates uncertainty, and for travel portals it creates an opportunity to provide fare forecasts and booking advice. **Predictive analytics** — the use of historical data, statistical techniques and machine learning to predict future or unknown outcomes — is well suited to this problem.

Machine learning regression models can learn the hidden patterns between flight characteristics and the final ticket price from a large set of past records. Once trained, such a model can estimate the fare of any new flight configuration within a fraction of a second.

1.2 Problem Statement

Given the details of a domestic flight in India — airline, source city, destination city, departure time, arrival time, number of stops, class of travel, duration of journey and the number of days left before departure — **predict the ticket price (in Indian Rupees)** as accurately as possible.

Since the target variable (price) is continuous, this is a **supervised regression problem**. The project also aims to identify which factors influence the fare the most and to compare the effectiveness of different regression algorithms on this task.

1.3 Objectives

The main objectives of the project are:

1. To collect a large, publicly available and reliable dataset of Indian domestic flight fares.
2. To clean and pre-process the data by handling redundant columns, duplicates and outliers.
3. To perform Exploratory Data Analysis (EDA) and find meaningful patterns and insights about airfare.

4. To engineer and encode features so that they can be used by machine learning models.
5. To build and train five regression models: Linear Regression, Ridge Regression, Decision Tree, Random Forest and XGBoost.
6. To evaluate and compare the models using R^2 score, MSE, RMSE and MAE, along with cross validation.
7. To identify the best model and deploy it as an interactive web-based demonstration.

1.4 Scope of the Project

The project covers economy and business class flights operated by six airlines (Vistara, Air India, Indigo, GO FIRST, AirAsia and SpiceJet) between six metro cities, forming 30 city-pair routes. The data relates to journeys with a booking window of 1 to 49 days. Predictions made by the model are therefore valid for these routes, airlines and booking windows. The methodology, however, is general and can be extended to more cities, airlines and time periods when more data becomes available.

1.5 Organisation of the Report

The rest of this report is organised as follows. **Chapter 2** reviews existing work on airfare prediction. **Chapter 3** describes the dataset in detail. **Chapter 4** explains the methodology, tools and evaluation metrics. **Chapter 5** covers data cleaning, and **Chapter 6** presents the exploratory data analysis. **Chapter 7** describes feature engineering and encoding. **Chapter 8** explains the five regression models used. **Chapter 9** presents and compares the results. **Chapter 10** describes the interactive web demonstration, and **Chapter 11** concludes the report with limitations and future scope.

CHAPTER 2
LITERATURE REVIEW

Airfare prediction has been studied by researchers for nearly two decades, both to help customers decide when to buy tickets and to help airlines and agencies understand pricing behaviour. This chapter summarises some of the important approaches reported in the literature.

2.1 Early Work on Airfare Prediction

One of the earliest and most cited studies is by **Etzioni et al. (2003)**, who built a system called *Hamlet* that observed fares over time and advised customers whether to "buy now" or "wait". Using a combination of rule learning, time-series methods and reinforcement learning, the system was able to save customers a substantial amount compared to buying immediately. This work showed that fare movements, although complex, contain learnable patterns.

Groves and Gini (2013) extended this idea by using partial least squares regression and feature selection on airline data to predict the optimal purchase time. They found that the number of days before departure and the day of the week were among the most useful predictors.

2.2 Machine Learning Approaches

With the availability of larger datasets, researchers started applying standard machine learning regressors. **Tziridis et al. (2017)** compared eight machine learning models — including Multilayer Perceptron, Random Forest, Regression Tree, Bagging Regression Tree and Support Vector Regression — for airfare prediction on European routes. Their results showed that ensemble tree-based methods such as Bagging Regression Trees achieved the highest accuracy, reaching close to 88%.

Rajankar and Sakharkar (2019) studied Indian flight fares using Linear Regression, Decision Tree, KNN and Random Forest, and reported that Random Forest performed best. Similar studies on the Indian market have repeatedly found that class of travel, airline, stops and days left before departure are the most important features.

2.3 Gradient Boosting Methods

Chen and Guestrin (2016) introduced **XGBoost**, a scalable and regularised implementation of gradient boosted decision trees. XGBoost has since become one of the most successful algorithms for structured (tabular) data and is widely used in industry and in data science

competitions. Several recent airfare studies report that boosting methods such as XGBoost, LightGBM and CatBoost outperform both linear models and single decision trees.

2.4 Research Gap and Contribution

Many existing studies use small datasets of a few thousand records or focus on routes outside India. This project uses a large dataset of over three lakh Indian domestic flight records and performs a systematic, side-by-side comparison of five regression models using four error metrics and cross validation. In addition, the best model is deployed as a working web application, which is often missing from academic studies.

Table 2.1: Summary of related work

Study	Data / Region	Methods	Key Finding
Etzioni et al. (2003)	US domestic routes	Rule learning, time series, Q-learning	Fare patterns are learnable; "buy or wait" advice saves money
Groves & Gini (2013)	US airline data	Partial least squares regression	Days to departure is a strong predictor
Chen & Guestrin (2016)	General tabular data	XGBoost (gradient boosting)	Fast, regularised boosting; state of the art on tabular data
Tziridis et al. (2017)	European routes	8 ML regressors	Ensemble trees performed best (~88%)
Rajankar & Sakharkar (2019)	Indian routes	LR, DT, KNN, RF	Random Forest performed best
This project (2026)	Indian metros, 3 lakh rows	LR, Ridge, DT, RF, XGBoost	XGBoost best with $R^2 = 0.9874$

CHAPTER 3
DATASET DESCRIPTION

3.1 Source of the Data

The dataset used in this project is the **"Flight Price Prediction"** dataset published on **Kaggle** by Shubham Bathwal (kaggle.com/datasets/shubhambathwal/flight-price-prediction). The data was collected by scraping the EaseMyTrip website, a popular Indian online travel portal, using the Octoparse web scraping tool. It contains information about flight booking options for travel between six major metro cities of India over a period of 50 days, from 11 February to 31 March 2022.

The dataset is freely and publicly available for research and educational purposes. The cleaned version of the file (**Clean_Dataset.csv**) was used, which contains **3,00,153 rows and 12 columns**.

3.2 Attribute Information

The table below describes each attribute present in the dataset.

Table 3.1: Description of dataset attributes

Attribute	Type	Description
Unnamed: 0	Integer	Row index (not a feature)
airline	Categorical (6)	Name of the airline: Vistara, Air India, Indigo, GO FIRST, AirAsia, SpiceJet
flight	Categorical (1,561)	Flight code, e.g. SG-8709
source_city	Categorical (6)	City from which the flight departs
departure_time	Categorical (6)	Time slot of departure: Early Morning, Morning, Afternoon, Evening, Night, Late Night
stops	Categorical (3)	Number of stops: zero, one, two_or_more
arrival_time	Categorical (6)	Time slot of arrival (same 6 slots)
destination_city	Categorical (6)	City where the flight lands
class	Categorical (2)	Seat class: Economy or Business
duration	Continuous	Total travel time in hours
days_left	Integer	Days between booking date and travel date (1–49)
price	Continuous	**Target variable** — ticket price in ₹

3.3 Descriptive Statistics

The numerical attributes of the raw dataset have the following summary statistics:

Table 3.2: Descriptive statistics of numerical attributes

Statistic	duration (hours)	days_left (days)	price (₹)
Count	3,00,153	3,00,153	3,00,153
Mean	12.22	26.00	20,889.66
Standard deviation	7.19	13.56	22,697.77
Minimum	0.83	1	1,105
25th percentile	6.83	15	4,783
Median	11.25	26	7,425
75th percentile	16.17	38	42,521
Maximum	49.83	49	1,23,071

The large difference between the mean (₹20,890) and the median (₹7,425) price indicates that the price distribution is highly skewed. This is due to the presence of business class tickets, which are far costlier than economy tickets.

3.4 Composition of the Data

Out of the 3,00,153 records, **68.85% are Economy** and **31.15% are Business** class tickets. Vistara (1,27,859 records) and Air India (80,892) together account for nearly 70% of the data, followed by Indigo (43,120), GO FIRST (23,173), AirAsia (16,098) and SpiceJet (9,011). Most flights have **one stop** (2,50,863 records), while 36,004 are non-stop and 13,286 have two or more stops. The source cities are well balanced, with Delhi (61,343) and Mumbai (60,896) having the most departures and Chennai (38,700) the fewest.

Table 3.3: Record count by category

Category	Values (number of records)
Class	Economy 2,06,666 · Business 93,487
Airline	Vistara 1,27,859 · Air India 80,892 · Indigo 43,120 · GO FIRST 23,173 · AirAsia 16,098 · SpiceJet 9,011
Stops	One 2,50,863 · Zero 36,004 · Two or more 13,286
Source city	Delhi 61,343 · Mumbai 60,896 · Bangalore 52,061 · Kolkata 46,347 · Hyderabad 40,806 · Chennai 38,700

CHAPTER 4
METHODOLOGY

4.1 Project Workflow

The project follows the standard predictive analytics life cycle. Each stage of the workflow is shown in Figure 4.1 and explained in the following chapters.

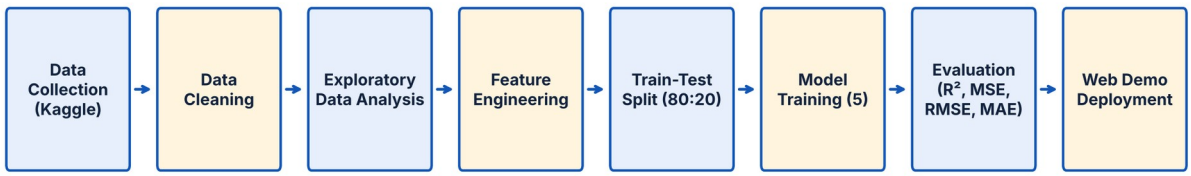


Figure 4.1: Workflow of the project

1. **Data Collection** — the dataset was downloaded from Kaggle in CSV format.
2. **Data Cleaning** — removal of unnecessary columns, duplicate rows and outliers; checking for missing and invalid values.
3. **Exploratory Data Analysis** — univariate, bivariate and multivariate analysis using statistical summaries and visualisations.
4. **Feature Engineering** — converting categorical attributes into numerical form using suitable encoding techniques.
5. **Train-Test Split** — dividing the data into 80% training and 20% testing sets, followed by feature scaling.
6. **Model Training** — training five regression models on the training set.
7. **Evaluation** — comparing models on the test set using R^2, MSE, RMSE and MAE, and validating with 5-fold cross validation.
8. **Deployment** — exporting the best model and building an interactive web page for real-time prediction.

4.2 Tools and Technologies

Table 4.1: Tools and libraries used

Tool / Library	Purpose
Python 3	Programming language for the complete analysis
Jupyter Notebook	Interactive development and documentation of code with outputs
Pandas	Loading, cleaning and manipulating tabular data
NumPy	Numerical computations and array operations
Matplotlib & Seaborn	Data visualisation and plotting
Scikit-learn	Pre-processing, Linear/Ridge/Tree/Forest models, metrics and cross validation
XGBoost	Gradient boosted decision tree regressor
HTML, CSS, JavaScript	Interactive web demonstration of the trained model
Git & GitHub	Version control and hosting of code and demo

4.3 Evaluation Metrics

Since price prediction is a regression task, classification accuracy cannot be used. Instead, the following four standard regression metrics are used, where y_i is the actual price, $\hat{y}_i$ is the predicted price, $\bar{y}$ is the mean of actual prices and n is the number of test samples.

4.3.1 Coefficient of Determination (R^2 Score)

R^2 measures the proportion of the variance in the target variable that is explained by the model. It ranges up to 1, where 1 means a perfect fit and 0 means the model is no better than always predicting the mean. In this report, the R^2 score is used as the **accuracy** of a regression model.

$$R^2 = 1 - [\ \Sigma\ (y_i - \hat{y}_i)^2\ ] / [\ \Sigma\ (y_i - \bar{y})^2\ ]$$

4.3.2 Mean Squared Error (MSE)

MSE is the average of the squared differences between actual and predicted values. Squaring gives higher weight to large errors, so MSE is sensitive to big mistakes. Its unit is the square of the target unit (₹²).

$$MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$$

4.3.3 Root Mean Squared Error (RMSE)

RMSE is the square root of MSE. It brings the error back to the same unit as the target (₹), making it easier to interpret.

$$RMSE = \sqrt{MSE}$$

4.3.4 Mean Absolute Error (MAE)

MAE is the average of the absolute differences between actual and predicted values. It tells us how many rupees the prediction is off by, on average, and is less sensitive to outliers than RMSE.

$$MAE = (1/n)\ \Sigma\ |\ y_i - \hat{y}_i\ |$$

For R^2, a higher value is better, while for MSE, RMSE and MAE, a lower value is better.

4.4 Cross Validation

To make sure that the results are not due to one lucky train-test split, **K-fold cross validation** with K = 5 was used. The training data is divided into five equal parts; each time the model is trained on four parts and tested on the remaining one. The average R^2 over the five folds gives a more reliable estimate of model performance. Due to the size of the dataset, cross validation was performed on a random sample of 50,000 training records.

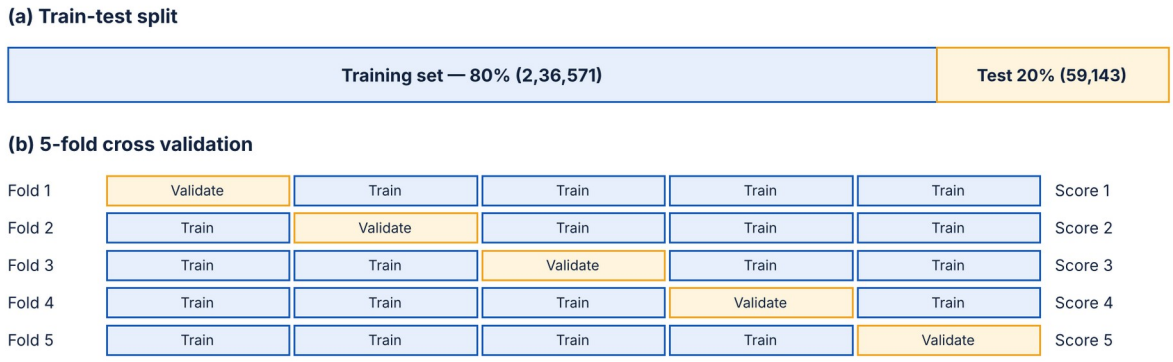


Figure 4.2: Train-test split and 5-fold cross validation scheme

CHAPTER 5
DATA CLEANING AND PRE-PROCESSING

Real-world data often contains irrelevant columns, missing values, duplicates and outliers that can reduce model performance. This chapter describes each cleaning step performed on the dataset.

5.1 Removing Unnecessary Columns

The column **Unnamed: 0** is only a row index created while saving the file and carries no information about the flight. The column **flight** contains the flight code (for example SG-8709 or UK-995) and has 1,561 unique values. The airline information is already present in the airline column, and keeping such a high-cardinality identifier can make the model memorise individual flights instead of learning general patterns. Both columns were therefore dropped, leaving 10 columns.

5.2 Handling Missing Values

The dataset was checked for missing values in every column using the *isnull()* function. **No missing values were found** in any of the columns, so no imputation was needed.

5.3 Removing Duplicate Records

After dropping the index and flight code columns, some rows became exact copies of each other. A total of **2,213 duplicate rows** were found and removed, because duplicates can bias the model and give an over-optimistic test score if the same record appears in both training and testing sets.

5.4 Checking Categorical Values

All the unique values of each categorical column were printed and checked for spelling mistakes, inconsistent case or invalid entries. All categories were found to be clean and consistent. It was also confirmed that no record has the same source and destination city.

5.5 Outlier Detection and Removal

Box plots were drawn for price, duration and days left to visually detect outliers (Figure 5.1).

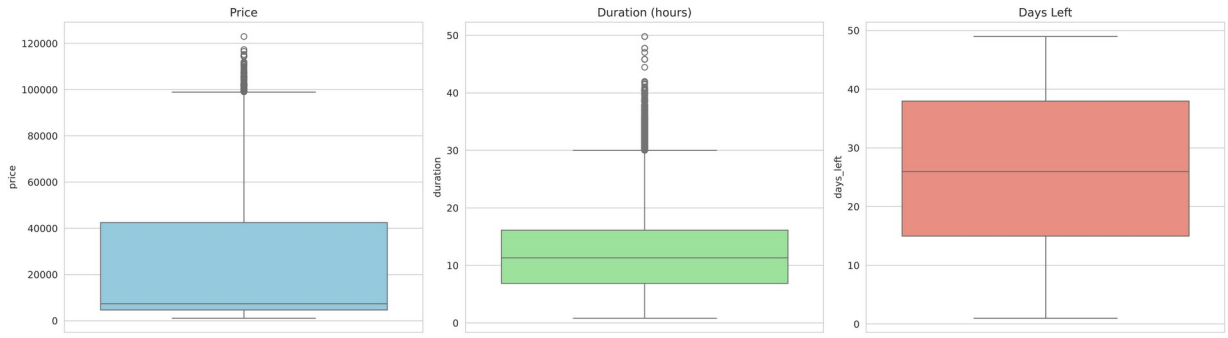


Figure 5.1: Box plots of price, duration and days left

The price column shows many points above the upper whisker. However, these high values belong to **business class** tickets and are genuine prices, not errors, so they were kept. The days left column has no outliers. The duration column has some extremely long journeys of more than 30 hours with multiple stops, which are rare and unusual. These were removed using the **Inter-Quartile Range (IQR)** method:

$$IQR = Q3 - Q1, \quad Upper\ limit = Q3 + 1.5 \times IQR$$

With Q1 = 6.83 hours and Q3 = 16.17 hours, the upper limit was found to be **30.05 hours**. All **2,226 records** with a duration greater than this limit were removed.

5.6 Summary of Data Cleaning

Table 5.1: Summary of data cleaning steps

Step	Action Taken	Rows Remaining
Original dataset	—	3,00,153
Drop columns	Removed Unnamed: 0 and flight	3,00,153
Missing values	None found	3,00,153
Duplicates	Removed 2,213 duplicate rows	2,97,940
Outliers (duration)	Removed 2,226 rows above 30.05 hours	**2,95,714**

After cleaning, the final dataset contains **2,95,714 records and 10 columns**, which was used for analysis and modelling.

CHAPTER 6
EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) helps in understanding the structure of the data, the distribution of variables and the relationships between the features and the target. The key analyses and their findings are presented in this chapter.

6.1 Distribution of Ticket Price

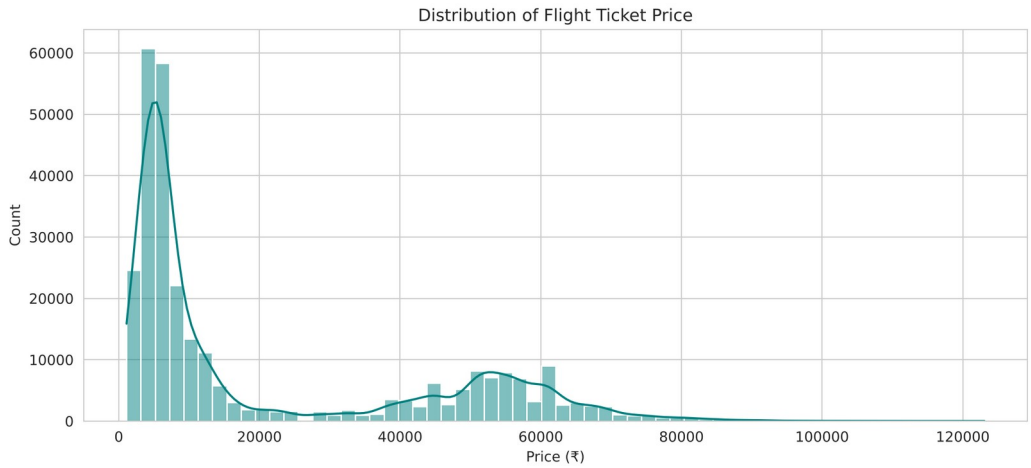


Figure 6.1: Distribution of flight ticket price

The price distribution is **bimodal** (it has two peaks). The first and larger peak lies between ₹2,000 and ₹15,000 and corresponds to economy class tickets, while the second peak between ₹40,000 and ₹70,000 corresponds to business class tickets. This shows that class is likely to be the single most important factor in deciding the price.

6.2 Economy vs Business Class

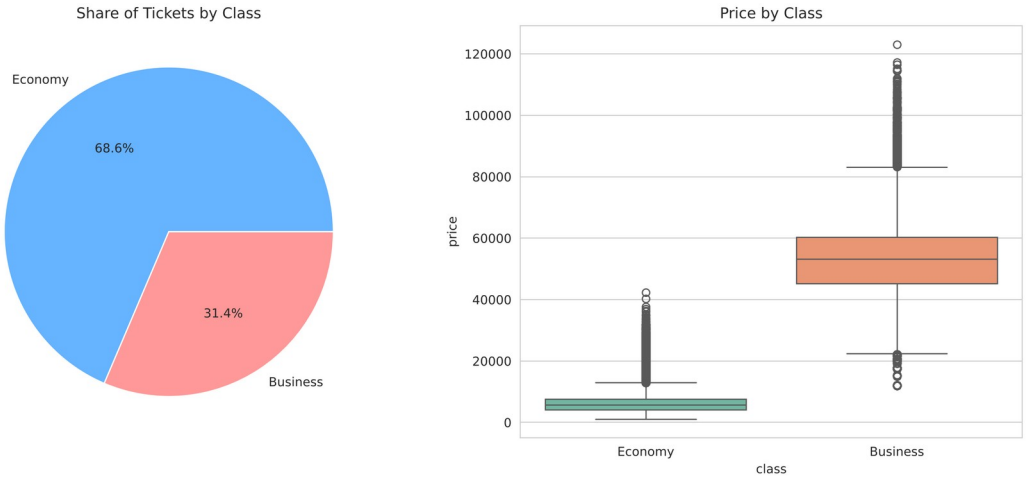


Figure 6.2: Share of tickets and price by class

About 69% of the tickets are economy and 31% are business class. The average economy fare is about **₹6,572** whereas the average business fare is about **₹52,540**, which means business class tickets are roughly **eight times costlier**. The ranges also hardly overlap, confirming that class divides the data into two very different price groups.

6.3 Airline-wise Analysis

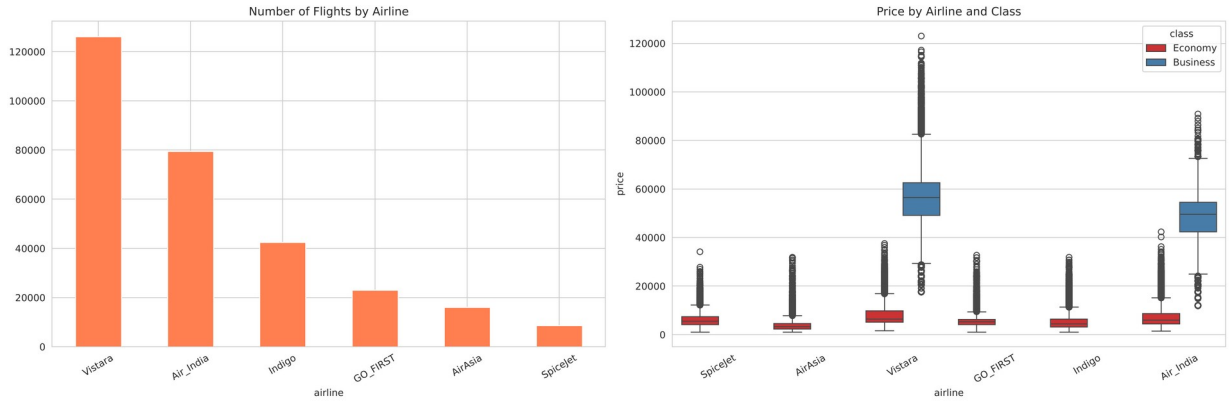


Figure 6.3: Number of flights and price by airline and class

Vistara and Air India operate the highest number of flights in the dataset and are the **only airlines that offer business class**. In economy class, **AirAsia (₹4,091) and Indigo (₹5,324)** are the cheapest on average, followed by GO FIRST and SpiceJet, while **Vistara (₹7,807) and Air India (₹7,314)** are the most expensive. Full-service carriers therefore charge more than low-cost carriers.

Table 6.1: Average economy fare by airline

Airline	Average Economy Fare (₹)	Type
AirAsia	4,091	Low-cost carrier
Indigo	5,324	Low-cost carrier
GO FIRST	5,652	Low-cost carrier
SpiceJet	6,179	Low-cost carrier
Air India	7,314	Full-service carrier
Vistara	7,807	Full-service carrier

6.4 Effect of Days Left before Departure

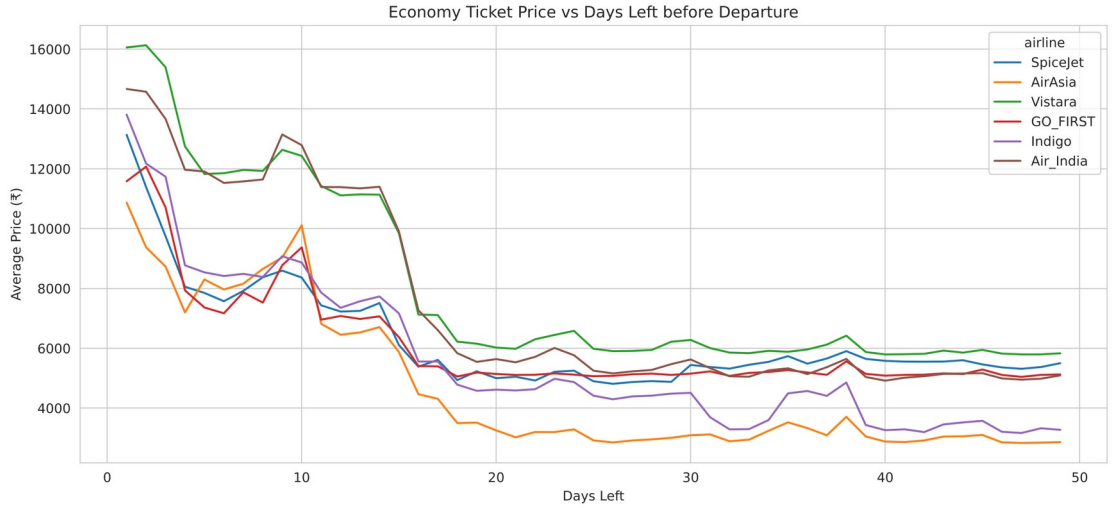


Figure 6.4: Economy ticket price vs days left before departure

This is one of the most important findings of the analysis. For all airlines, the fare stays fairly low and stable when the ticket is booked **more than about 15–20 days in advance**. As the departure date comes closer, prices rise steadily, and in the **last 1–3 days** they jump sharply. On average, economy tickets booked 1–3 days before travel cost about ₹13,808, compared to about ₹4,919 for tickets booked 31–49 days before — nearly **2.8 times more**. This reflects the dynamic pricing strategy of airlines, which charge last-minute travellers (often business travellers) higher prices.

Table 6.2: Average economy fare by booking window

Days Left before Travel	Average Economy Fare (₹)
1–3 days	13,808
4–7 days	10,565
8–14 days	10,048
15–30 days	5,588
31–49 days	4,919

6.5 Effect of Number of Stops



Figure 6.5: Average price by number of stops

Surprisingly, **non-stop flights are the cheapest** on average (about ₹4,013 in economy), while one-stop (₹6,813) and two-or-more-stop flights (₹9,142) are costlier. This is because non-stop routes are mostly served by low-cost carriers, while connecting itineraries are mostly sold by full-service airlines and combine the price of two flight segments.

6.6 Departure and Arrival Time

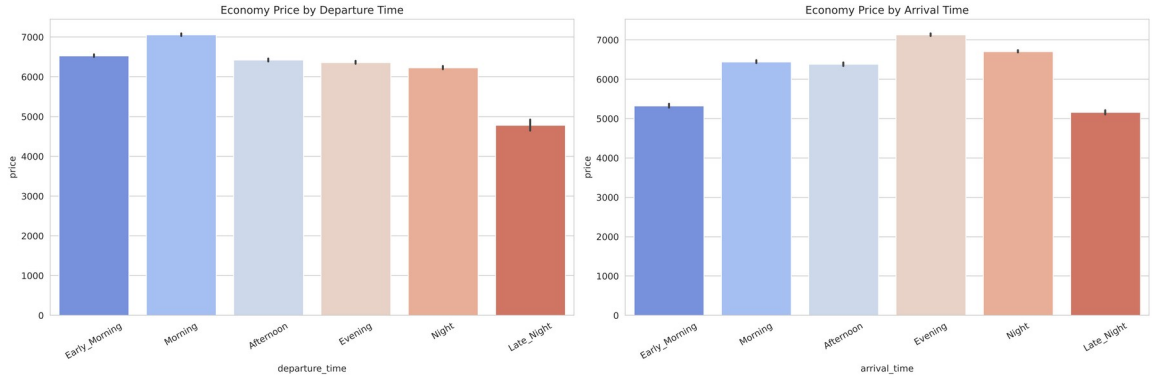


Figure 6.6: Economy price by departure and arrival time

Late night flights are clearly the cheapest, with an average economy fare of about ₹4,785, because they are less convenient for most passengers. Morning departures are the most expensive (about ₹7,119). The arrival time shows a similar pattern, with late night and early morning arrivals being cheaper.

6.7 Route-wise Analysis

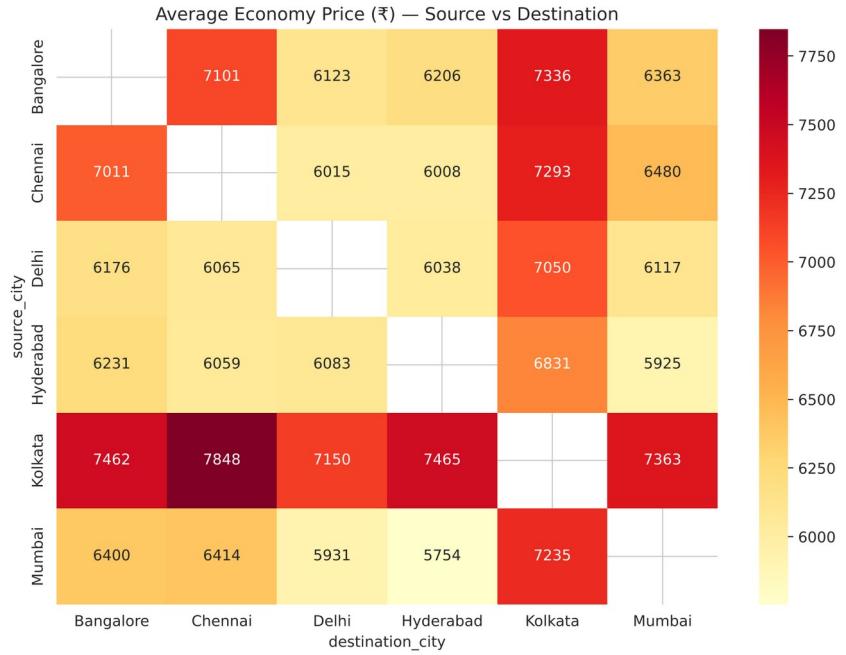


Figure 6.7: Average economy price by source and destination city

The heat map shows the average economy fare for each of the 30 routes. Routes starting from or ending at **Delhi** are generally the cheapest, likely due to high competition and frequency. Routes involving **Chennai, Bangalore and Kolkata** tend to be costlier. The fare between two cities is roughly similar in both directions.

6.8 Duration vs Price

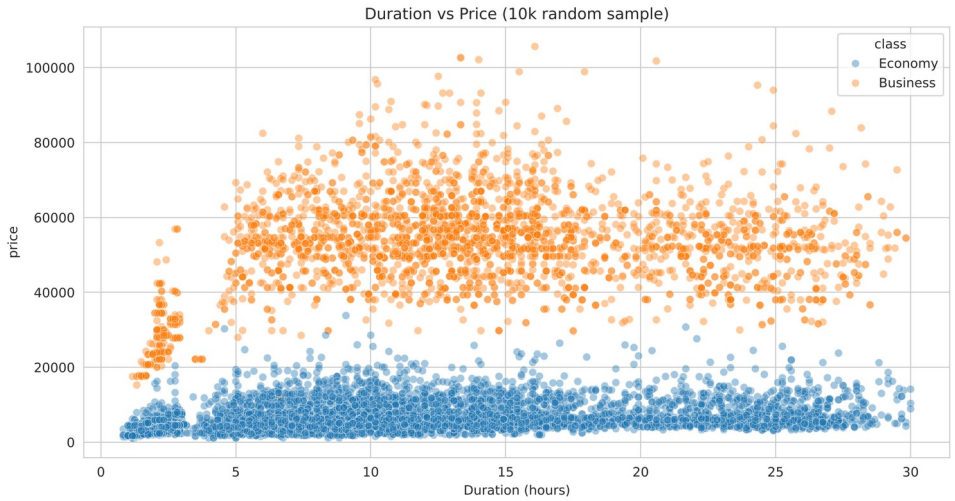


Figure 6.8: Duration vs price (random sample of 10,000 flights)

There is no simple straight-line relationship between duration and price. Within each class, prices vary widely for the same duration. Very short flights are mostly non-stop low-cost flights and are cheaper, while medium durations (connecting flights) show a wider spread.

This non-linear behaviour suggests that tree-based models will perform better than linear models.

6.9 Key Insights from EDA

- **Class** is the strongest factor — business fares are about 8 times economy fares.
- **Booking early saves money** — fares rise sharply in the last two weeks before departure.
- **Low-cost carriers** (AirAsia, Indigo) are cheaper than full-service carriers (Vistara, Air India).
- **Non-stop and late night flights** are generally the cheapest options.
- **Route** matters — Delhi routes are cheaper, while Chennai and Bangalore routes are costlier.
- Most relationships are **non-linear**, so advanced tree-based models are expected to perform best.

CHAPTER 7
FEATURE ENGINEERING

Machine learning models work with numbers, so all categorical attributes have to be converted into a numerical form. Different encoding techniques were chosen based on the nature of each attribute.

7.1 Binary Encoding of Class

The class attribute has only two values, so it was mapped directly to **Economy = 0** and **Business = 1**.

7.2 Ordinal Encoding of Stops

The stops attribute has a natural order (zero < one < two or more). It was therefore encoded as **zero = 0, one = 1 and two_or_more = 2**, preserving this order.

7.3 One-Hot Encoding of Nominal Attributes

The attributes airline, source_city, destination_city, departure_time and arrival_time are **nominal** — their values have no natural order. Assigning numbers like 1, 2, 3 to them would wrongly suggest an order, so **one-hot encoding** was used. Each category becomes a separate binary column (0 or 1). To avoid the *dummy variable trap* (perfect multicollinearity) in linear models, the first category of each attribute was dropped using *drop_first = True*.

airline
Vistara
Indigo
AirAsia
Air_India

one-hot →

Air_India	GO_FIRST	Indigo	SpiceJet	Vistara
0	0	0	0	**1**
0	0	**1**	0	0
0	0	0	0	0
1	0	0	0	0

AirAsia becomes the reference category (all zeros) because drop_first = True

Figure 7.1: Example of one-hot encoding for the airline attribute

Table 7.1: Encoding applied to each attribute

Attribute	Encoding Technique	Resulting Columns
class	Binary (0 / 1)	1
stops	Ordinal (0 / 1 / 2)	1
airline	One-hot (drop first)	5
source_city	One-hot (drop first)	5
destination_city	One-hot (drop first)	5
departure_time	One-hot (drop first)	5
arrival_time	One-hot (drop first)	5
duration	Kept as numeric	1
days_left	Kept as numeric	1
Total input features		**29**

After encoding, the dataset had **30 columns** — 29 input features and 1 target variable (price).

7.4 Correlation with Price

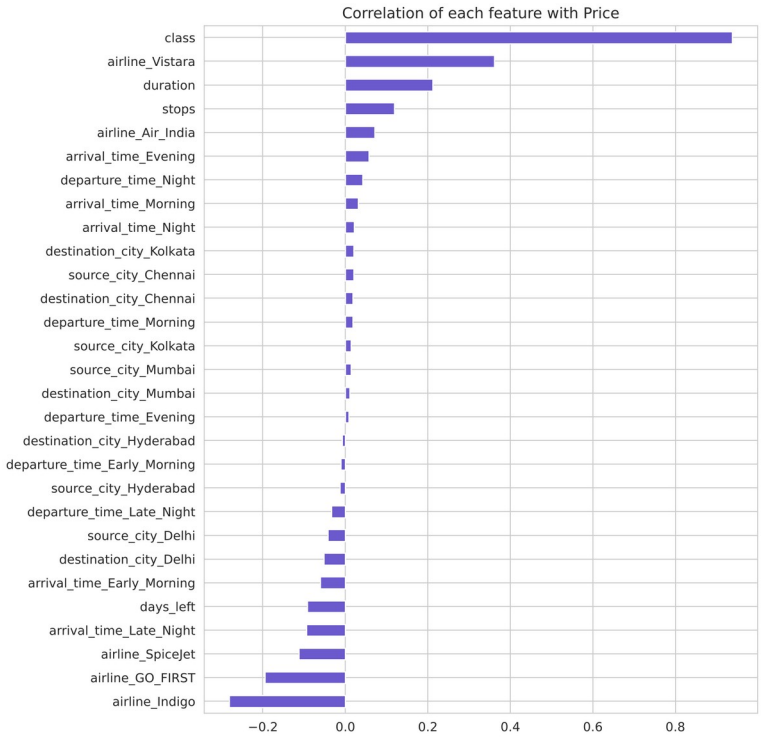


Figure 7.2: Correlation of each encoded feature with price

The correlation chart confirms the EDA findings. **Class has an extremely high positive correlation (about 0.94)** with price. Vistara and Air India have positive correlations, while AirAsia and Indigo are negatively correlated with price. Days left has a negative correlation,

meaning that more days left leads to a lower price. Stops and duration show a moderate positive correlation.

7.5 Train-Test Split and Feature Scaling

The data was split into a **training set of 80% (2,36,571 records)** and a **testing set of 20% (59,143 records)** using *train_test_split* with *random_state = 42* for reproducibility. The features were then standardised using **StandardScaler**, which transforms each feature to have zero mean and unit variance:

$$z = (x - \mu) / \sigma$$

The scaler was fitted only on the training data and then applied to the test data, to avoid any information from the test set leaking into training. Scaling is important for Linear and Ridge Regression, while tree-based models are not affected by it.

CHAPTER 8
MODEL BUILDING

Five regression algorithms were selected so that the comparison covers simple linear models, a single tree model and two powerful ensemble models. This chapter explains how each algorithm works and the settings used.

8.1 Linear Regression

Linear Regression is the simplest regression technique. It assumes that the target is a linear combination of the input features and finds the coefficients that minimise the sum of squared errors (Ordinary Least Squares).

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

It is fast, easy to interpret and serves as a good **baseline model**. However, it cannot capture non-linear relationships or interactions between features unless they are explicitly created.

8.2 Ridge Regression

Ridge Regression is a regularised version of linear regression. It adds an **L2 penalty** on the size of the coefficients to the loss function, which reduces overfitting and handles multicollinearity among features.

$$Loss = \Sigma\,(y_i - \hat{y}_i)^2 + \alpha\,\Sigma\,\beta_j^2$$

Here α controls the strength of regularisation. A value of α = 1.0 was used. Like linear regression, Ridge remains a linear model.

8.3 Decision Tree Regressor

A Decision Tree splits the data into smaller and smaller groups by asking a series of yes/no questions on the features (for example, "Is class = Business?" or "Is days_left < 15?"). At each node, the split that most reduces the variance (mean squared error) of the target is chosen. The prediction at a leaf is the average price of the training samples in that leaf.

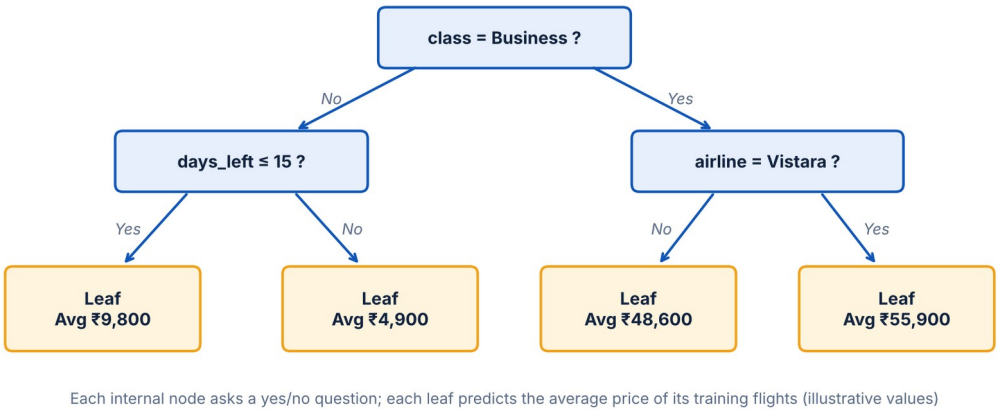


Figure 8.1: Structure of a regression decision tree (illustrative)

Decision trees can model non-linear relationships and interactions naturally and are easy to visualise. However, a single deep tree can easily overfit. To control this, the tree depth was limited to **20** and each leaf had to contain at least **5 samples**.

8.4 Random Forest Regressor

Random Forest is an **ensemble** of many decision trees built using *bagging* (bootstrap aggregation). Each tree is trained on a random sample of the training data, and at each split only a random subset of features is considered. The final prediction is the average of all trees.

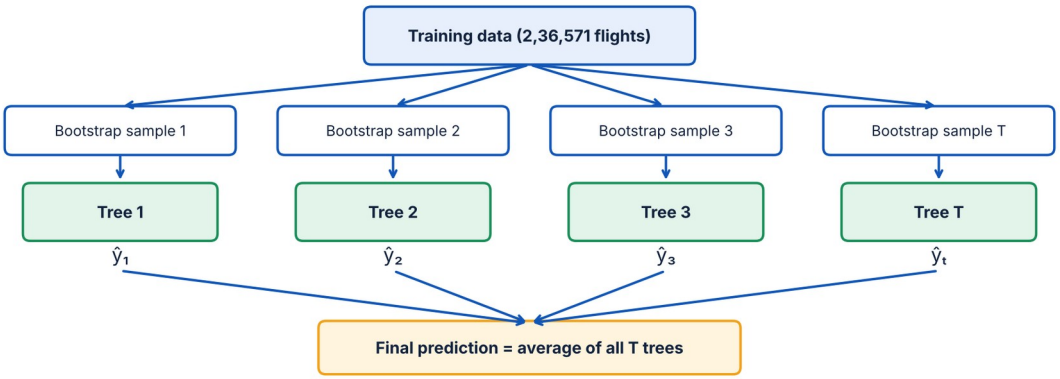


Figure 8.2: Working of the Random Forest (bagging) regressor

$$\hat{y} = (1/T)\ \Sigma\ \hat{y}_t\ ,\ \ t = 1 \ldots T$$

Averaging many different trees reduces variance and overfitting, giving much more stable and accurate predictions than a single tree. A forest of **60 trees** with a maximum depth of **25** and a minimum of 2 samples per leaf was used.

8.5 XGBoost Regressor

XGBoost (Extreme Gradient Boosting) is also an ensemble of decision trees, but the trees are built **sequentially** using *boosting*. Each new tree is trained to correct the errors (residuals)

made by the previous trees. The final prediction is the sum of the predictions of all trees, each scaled by a learning rate η.

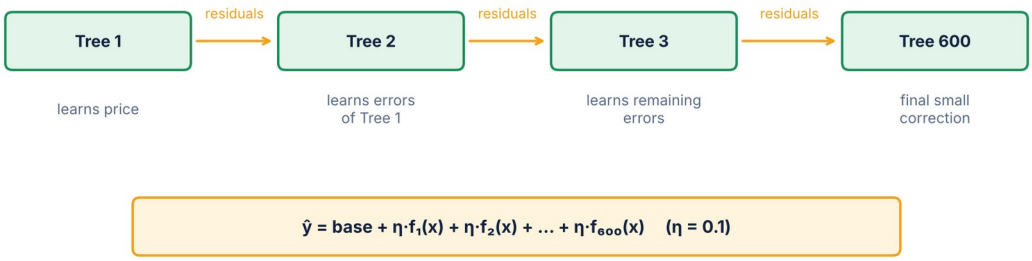


Figure 8.3: Sequential boosting in XGBoost — each tree corrects the previous errors

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot f_t(x)$$

XGBoost also includes regularisation terms in its objective function, handles large datasets efficiently and uses second-order gradient information for faster convergence. It is widely regarded as one of the best algorithms for tabular data. **600 trees** with a learning rate of **0.1** and a maximum depth of **8** were used.

8.6 Hyperparameters Used

Table 8.1: Hyperparameters of the five models

Model	Library	Key Hyperparameters
Linear Regression	scikit-learn	Default (ordinary least squares)
Ridge Regression	scikit-learn	alpha = 1.0
Decision Tree	scikit-learn	max_depth = 20, min_samples_leaf = 5
Random Forest	scikit-learn	n_estimators = 60, max_depth = 25, min_samples_leaf = 2
XGBoost	xgboost	n_estimators = 600, learning_rate = 0.1, max_depth = 8

All models used *random_state = 42* wherever applicable so that the results can be reproduced. Each model was trained on the scaled training data and then used to predict prices for the test data.

CHAPTER 9
RESULTS AND DISCUSSION

9.1 Model Comparison Table

The performance of all five models on the test set of 59,143 flights is shown in Table 9.1. The models are ranked by their test R^2 score.

Table 9.1: Performance comparison of the five regression models

Rank	Model	Train R^2	Test R^2 (Accuracy)	MSE	RMSE (₹)	MAE (₹)
1	**XGBoost**	**0.9909**	**0.9874 (98.74%)**	**65,38,317**	**2,557**	**1,342**
2	Random Forest	0.9941	0.9864 (98.64%)	70,46,288	2,654	1,119
3	Decision Tree	0.9869	0.9811 (98.11%)	97,88,739	3,129	1,419
4	Linear Regression	0.9102	0.9096 (90.96%)	4,67,61,203	6,838	4,536
5	Ridge Regression	0.9102	0.9096 (90.96%)	4,67,61,199	6,838	4,536

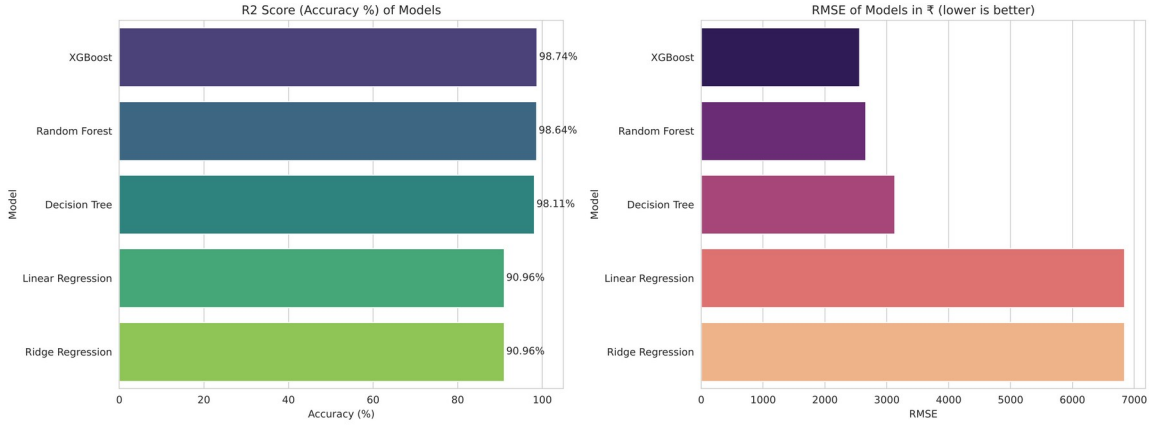


Figure 9.1: R^2 score (accuracy) and RMSE of the five models

9.2 Discussion of Results

9.2.1 Linear and Ridge Regression

Both linear models achieved an **R^2 of 0.9096**, meaning they explain about 91% of the variation in price. This is mainly because the class feature alone separates economy and business fares very well. However, their RMSE of ₹6,838 and MAE of ₹4,536 are very high compared to the average economy fare of about ₹6,500. Ridge Regression gave almost identical results to Linear Regression, because the dataset is large and the features are not heavily collinear, so regularisation has little effect. The train and test scores are almost equal, which means these models are **underfitting** — they are too simple to capture the non-linear effects of days left, duration and route.

9.2.2 Decision Tree

The Decision Tree improved the R^2 to **0.9811** and reduced the RMSE by more than half to ₹3,129. This shows the benefit of modelling non-linear relationships and interactions such as "business class on Vistara booked within 3 days".

9.2.3 Random Forest

Random Forest achieved an R^2 of **0.9864** and the **lowest MAE of ₹1,119**, meaning its typical error is the smallest. Its training R^2 (0.9941) is slightly higher than the test R^2, indicating mild overfitting, but the test performance is still excellent.

9.2.4 XGBoost

XGBoost gave the best overall performance, with the highest test R^2 of **0.9874** and the lowest MSE (65,38,317) and RMSE (₹2,557). The gap between its training and test R^2 is the smallest among the tree models (0.0035), showing good generalisation. Since RMSE penalises large errors more, XGBoost's lowest RMSE shows that it makes fewer large mistakes, especially on expensive business class tickets. It was therefore selected as the **final model**.

9.3 Cross Validation Results

Table 9.2: 5-fold cross validation R^2 scores (50,000-record sample)

Model	Mean CV R^2	Standard Deviation
Linear Regression	0.9098	0.0018
Ridge Regression	0.9098	0.0018
Decision Tree	0.9695	0.0012
Random Forest	0.9785	0.0011
XGBoost	**0.9795**	**0.0009**

The cross validation scores follow the same ranking as the test scores, with XGBoost again at the top. The standard deviations are very small (below 0.002), showing that the models perform consistently across different subsets of the data. The CV scores of the tree models are slightly lower than their test scores only because they were trained on a smaller sample of 50,000 records instead of the full training set.

9.4 Actual vs Predicted Prices

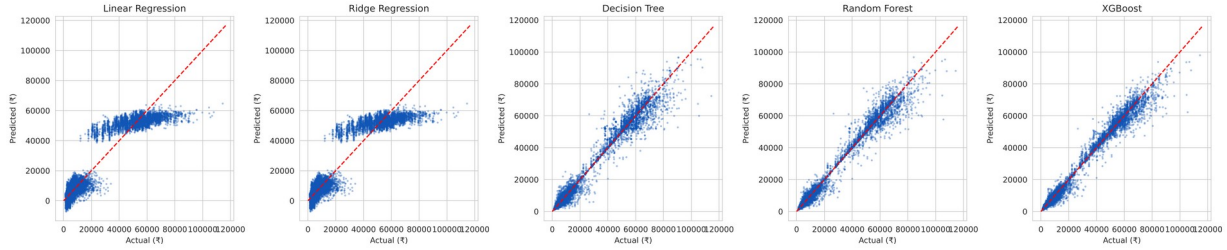


Figure 9.2: Actual vs predicted prices for all five models

In Figure 9.2, each point is a test flight and the red dashed line represents perfect prediction. For the linear models the points are widely scattered, and some predictions are even negative for cheap economy flights. The tree-based models have points tightly clustered around the line for both economy and business classes.

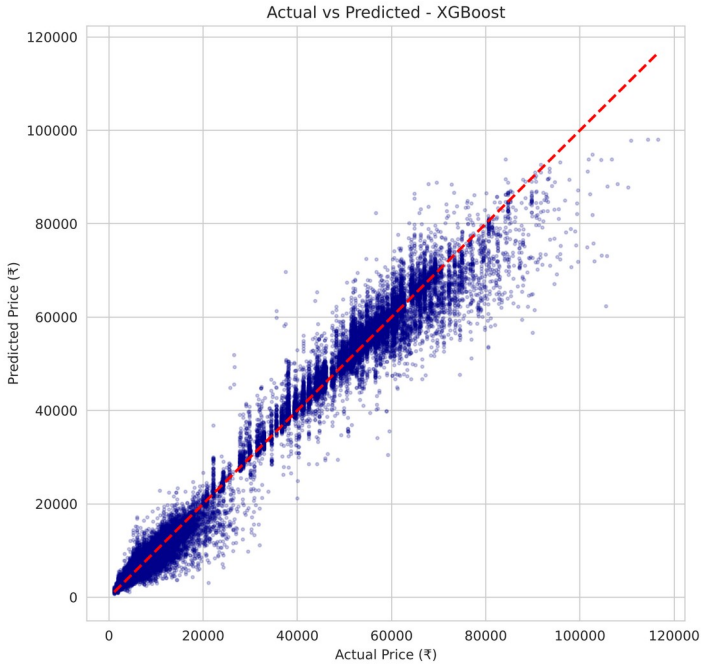


Figure 9.3: Actual vs predicted prices — XGBoost

9.5 Residual Analysis

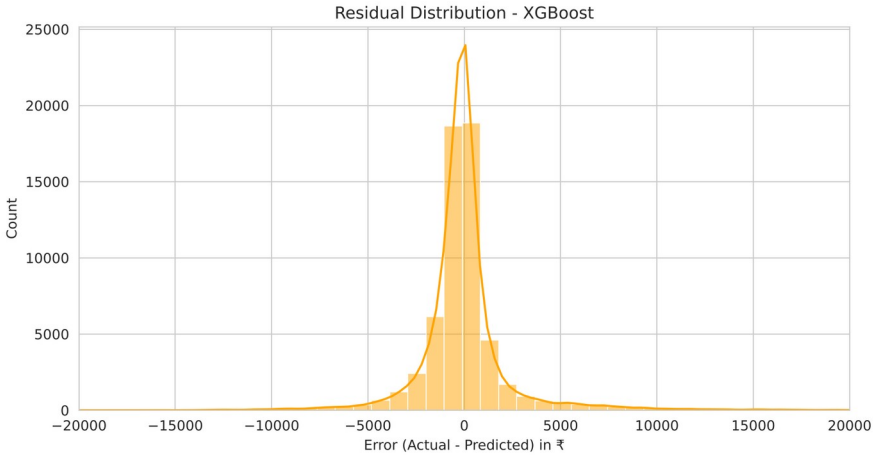


Figure 9.4: Distribution of residuals — XGBoost

The residuals (actual minus predicted price) of XGBoost are centred around zero and follow a roughly symmetric, bell-shaped distribution. Most errors lie within ±₹2,500. This indicates that the model has no systematic bias of over- or under-predicting prices.

9.6 Feature Importance

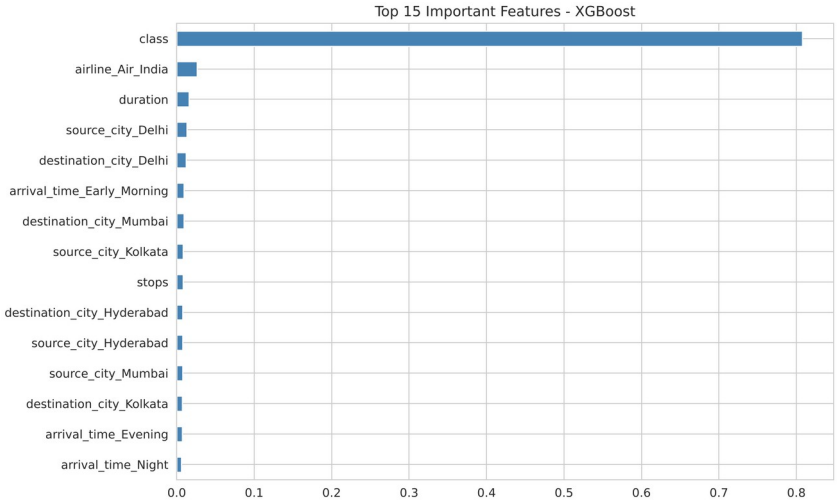


Figure 9.5: Top 15 important features — XGBoost

According to XGBoost, **class** is by far the most important feature, followed by airline (especially Vistara and Air India), the source and destination cities, **days left**, duration and stops. This matches the findings of the EDA and the correlation analysis.

9.7 Sample Predictions

Table 9.3: Sample predictions of the XGBoost model on test data

S. No.	Actual Price (₹)	Predicted Price (₹)	Difference (₹)
1	4,629	5,127	−498
2	12,166	11,778	388
3	2,022	2,671	−649
4	2,150	2,150	0
5	60,260	61,120	−860
6	2,250	2,772	−522
7	50,271	50,231	40
8	74,844	71,792	3,052

The predicted prices are very close to the actual prices for both cheap economy tickets and expensive business tickets, which confirms the high accuracy of the model.

CHAPTER 10
WEB-BASED DEMONSTRATION

To make the model useful and easy to demonstrate, an interactive web page named **FareCast India** was developed. It allows any user to enter flight details and instantly see the predicted ticket price.

10.1 Model Export

An XGBoost model with 300 trees (maximum depth 8) was trained on the same cleaned and encoded data; it achieved a test R^2 of 0.9845. Since tree models do not require feature scaling, this model was trained on unscaled features. All the trees — split features, thresholds and leaf values — were exported into a compact JSON file (*model.json*, about 1.9 MB). A small JavaScript function walks through every tree for the given input and adds up the leaf values, exactly reproducing the predictions of the Python model (maximum difference below ₹0.1). As a result, the prediction runs **entirely in the browser** without needing any server.

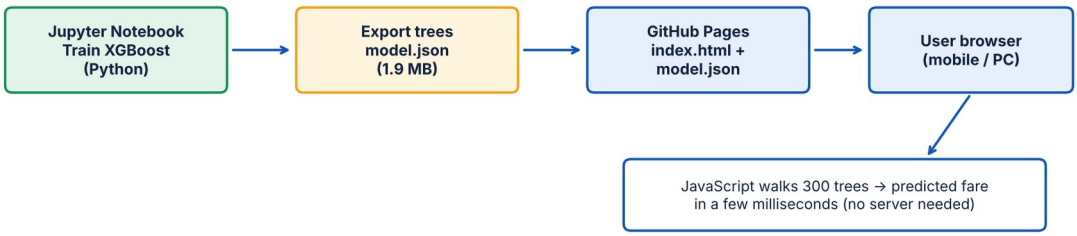


Figure 10.1: Architecture of the web-based demonstration

10.2 Features of the Web Page

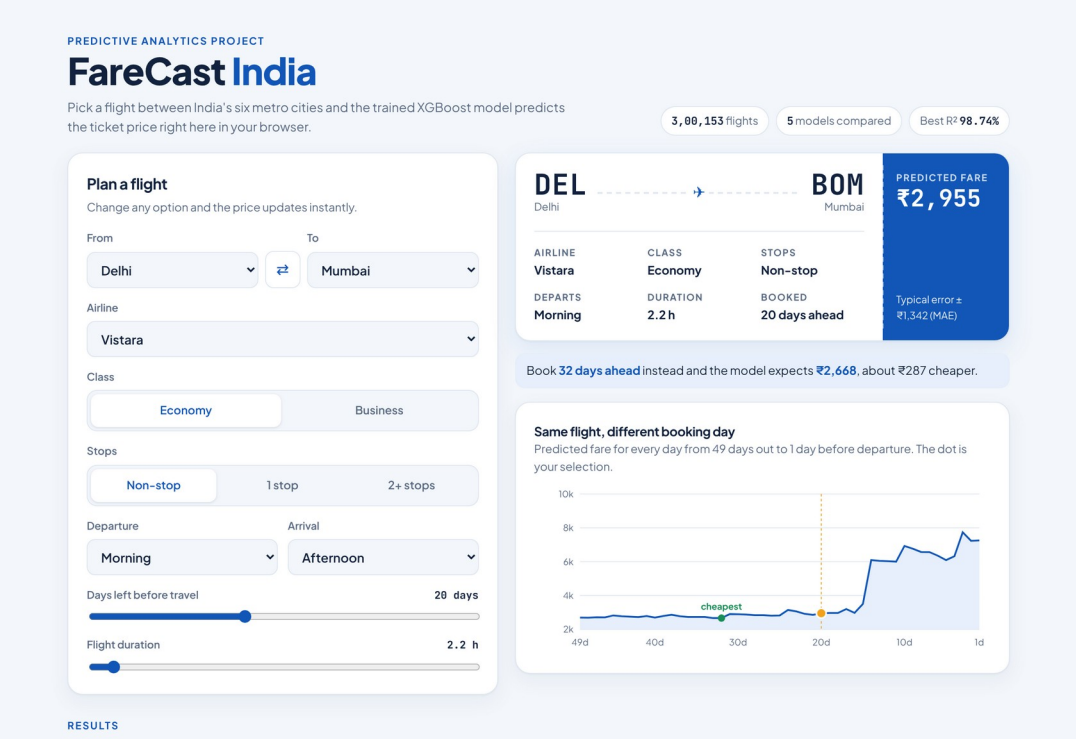


Figure 10.2: FareCast India web page — desktop view

- **Flight planner form** — select source and destination city, airline, class, stops, departure and arrival time, days left before travel and flight duration.
- **Boarding-pass style result** — shows the route with airport codes (DEL, BOM, BLR, CCU, HYD, MAA) and the predicted fare, updated instantly on every change.
- **Booking-day chart** — plots the predicted fare for the same flight for every booking day from 49 days to 1 day before departure and marks the cheapest day.
- **Smart suggestion** — tells the user how much they could save by booking on the cheapest day.
- **Results and insights** — shows the five-model comparison table and charts of average fare by airline and booking window.

10.3 Deployment

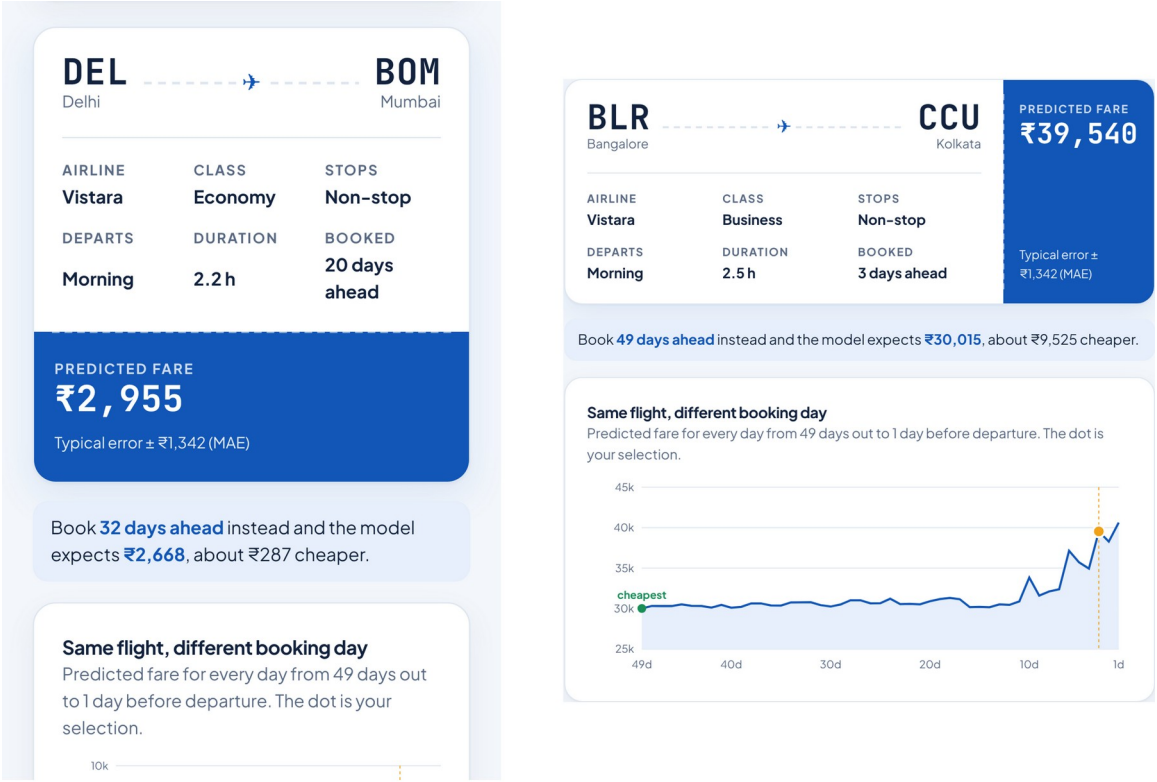
The web page (*docs/index.html*) and the model file are stored in the project's GitHub repository and are served using **GitHub Pages**, so the demonstration can be opened on any mobile phone or computer through a simple link. The design is responsive and works on small screens, as shown in Figure 10.3(a).

10.4 Example Use Case

For example, for an economy Vistara non-stop flight from Delhi to Mumbai departing in the morning, the model predicts a fare of about ₹7,259 when booked one day before travel, but

only about ₹2,955 when booked 20 days before. The chart on the page makes this difference immediately visible to the user.

Figure 10.3(b) shows a second example: a **Vistara business class non-stop flight from Bangalore to Kolkata booked 3 days before travel**. The model predicts **₹39,540**, and the chart shows that booking 49 days in advance would cost about ₹30,015 — a saving of about ₹9,525. This demonstrates the practical value of predictive analytics for travellers.



(a) Mobile view **(b) Business class example**

Figure 10.3: (a) Mobile view of the web page, (b) prediction for a business class flight from Bangalore to Kolkata

CHAPTER 11
CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

In this project, a complete predictive analytics pipeline was developed to predict the price of domestic flight tickets in India. A large public dataset of 3,00,153 flight records from Kaggle was cleaned, analysed and used to train five regression models. The main conclusions are:

1. After removing redundant columns, 2,213 duplicates and 2,226 outliers, a clean dataset of 2,95,714 records was obtained.
2. EDA showed that **class** is the most important factor (business fares are ~8× economy fares), that fares **rise sharply in the last two weeks** before departure, and that low-cost carriers, non-stop flights and late-night flights are cheaper.
3. Linear models achieved an R^2 of 0.9096 but had high errors, showing that the problem is non-linear.
4. Tree-based models performed much better. **XGBoost was the best model with an R^2 score of 0.9874 (98.74%), RMSE of ₹2,557 and MAE of ₹1,342**, followed by Random Forest (98.64%) and Decision Tree (98.11%).
5. 5-fold cross validation confirmed that the results are stable and reliable.
6. The final model was deployed as an interactive web page that predicts fares in real time and suggests the cheapest booking day.

The project demonstrates that machine learning, especially gradient boosting, can predict airfares with very high accuracy and can provide useful guidance to travellers.

11.2 Limitations

- The dataset covers only **six metro cities (30 routes)** and six airlines, so the model cannot predict fares for other cities or airlines.
- The data was collected over a short period (February–March 2022) and does not capture seasonal effects such as festivals, holidays or summer vacations.
- Airline-specific factors such as remaining seats, fuel prices and promotional offers are not available in the dataset.
- Some airlines in the data (for example GO FIRST) have since stopped operations, and Vistara has merged with Air India, so the market has changed since the data was collected.

11.3 Future Scope

- Collect data for more cities, airlines and a longer time period, including festival and holiday seasons.
- Add new features such as day of the week, public holidays, fuel price and seat availability.
- Perform hyperparameter tuning using GridSearchCV or Bayesian optimisation to further improve accuracy.
- Try other advanced algorithms such as LightGBM, CatBoost and deep learning models.
- Build a live system that scrapes current fares daily and updates the model automatically.
- Extend the web application into a full fare-alert service that notifies users when prices are expected to drop.

REFERENCES

[1] Bathwal, S. (2022). *Flight Price Prediction* [Dataset]. Kaggle. https://www.kaggle.com/datasets/shubhambathwal/flight-price-prediction

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

[3] Etzioni, O., Tuchinda, R., Knoblock, C. A., & Yates, A. (2003). To buy or not to buy: Mining airfare data to minimize ticket purchase price. *Proceedings of the 9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 119–128.

[4] Groves, W., & Gini, M. (2013). Optimal airline ticket purchasing using automated user-guided feature selection. *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, 150–156.

[5] Tziridis, K., Kalampokas, T., Papakostas, G. A., & Diamantaras, K. I. (2017). Airfare prices prediction using machine learning techniques. *25th European Signal Processing Conference (EUSIPCO)*, 1036–1039.

[6] Rajankar, S., & Sakharkar, N. (2019). A survey on flight pricing prediction using machine learning. *International Journal of Engineering Research & Technology (IJERT)*, 8(6).

[7] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[8] Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.

[9] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[10] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

[11] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

[12] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

APPENDIX A: SOURCE CODE

The complete code is available in the Jupyter Notebook *Flight_Price_Prediction.ipynb* in the project repository (github.com/UtkarshMehta7/house-price-prediction-ml). The key parts of the code are given below.

A.1 Importing Libraries and Loading Data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

df = pd.read_csv('Clean_Dataset.csv')
print("Shape of data:", df.shape)
```

A.2 Data Cleaning

```
df = df.drop(['Unnamed: 0', 'flight'], axis=1)
print(df.isnull().sum())
df = df.drop_duplicates()

Q1 = df['duration'].quantile(0.25)
Q3 = df['duration'].quantile(0.75)
IQR = Q3 - Q1
upper = Q3 + 1.5 * IQR
df = df[df['duration'] <= upper]
print("Final shape:", df.shape)
```

A.3 Feature Engineering

```
df['class'] = df['class'].map({'Economy': 0, 'Business': 1})
df['stops'] = df['stops'].map({'zero': 0, 'one': 1, 'two_or_more': 2})

df = pd.get_dummies(df, columns=['airline', 'source_city',
'destination_city',
                                 'departure_time', 'arrival_time'],
                    drop_first=True, dtype=int)

X = df.drop('price', axis=1)
y = df['price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

A.4 Model Training and Evaluation

```
models = {
```

```
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Decision Tree': DecisionTreeRegressor(max_depth=20, min_samples_leaf=5,
random_state=42),
    'Random Forest': RandomForestRegressor(n_estimators=60, max_depth=25,
min_samples_leaf=2,
                                           random_state=42, n_jobs=-1),
    'XGBoost': XGBRegressor(n_estimators=600, learning_rate=0.1, max_depth=8,
random_state=42)
}

results = []
for name, model in models.items():
    model.fit(X_train_sc, y_train)
    y_pred = model.predict(X_test_sc)
    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    results.append([name, r2, mse, rmse, mae])

results_df = pd.DataFrame(results, columns=['Model', 'R2', 'MSE', 'RMSE',
'MAE'])
results_df.sort_values('R2', ascending=False)
```

A.5 Cross Validation

```
idx = np.random.RandomState(42).choice(len(X_train_sc), 50000, replace=False)
X_cv, y_cv = X_train_sc[idx], y_train.iloc[idx]
kf = KFold(n_splits=5, shuffle=True, random_state=42)
for name, model in models.items():
    scores = cross_val_score(model, X_cv, y_cv, cv=kf, scoring='r2')
    print(name, scores.mean().round(4), scores.std().round(4))
```

A.6 Browser Prediction Function (JavaScript)

```
function predict(s) {
  const x = vec(s);            // encode the selected options
  let y = M.base;              // base score of the model
  for (const [F, T, L, R] of M.trees) {
    let i = 0;
    while (F[i] >= 0) i = x[F[i]] < T[i] ? L[i] : R[i];  // walk the tree
    y += T[i];                 // add the leaf value
  }
  return y;
}
```